Developer Portfolio Edition

Welcome to your implementation guide.

WINDOWS XP PORTFOLIO

Step-by-Step Technical Implementation Guide

A comprehensive manual to building a fully interactive, retro-themed developer portfolio replicating the classic Windows XP Luna desktop interface. Crafted with Next.js, Firebase Auth & Firestore, and GSAP animations.

AUTHOR: Kamal Kumar
VERSION: 1.0.0
DATE: June 13, 2026

Chapter 1: Project Architecture & Lifecycle

The Windows XP Portfolio is structured as a client-side state machine packaged in a modern Next.js single-page application. Replicating a desktop GUI inside a web browser requires high control over DOM rendering, absolute positioning coordinates, stacking layers (Z-indices), and animation execution. The application operates under a three-phase state lifecycle:

1. The Lifecycle States

- Boot Phase: Displays the retro Windows XP splash logo and tagline, simulating an authentic system booting process. An animated loading progress bar moves iteratively using CSS keyframe animations. The boot screen can be skipped by clicking anywhere on the viewport.
- Login Phase: Mimics the classic blue Windows XP account selection layout. Users are presented with accounts (such as an Administrator account and options to log in via credentials or OAuth Google authentication). Guest access is supported, enabling immediate read-only access.
- Desktop Phase: The main dashboard shell. Once loaded, the browser renders the classic Bliss wallpaper, draggable/resizable windows, floating desktop shortcuts, a fully functional taskbar with system tray/clock, and a dual-column Start Menu.

2. Component Stacking Hierarchy

To coordinate components and state transitions, the layout depends on a centralized orchestrator component (XPDesktop.jsx). Below is the core architectural topology:

```
src/
% % %  app/
%    % % %  layout.js          # Root layout, Google fonts (Arimo, Material Icons)
%    % % %  page.js            # Entry page importing XPDesktop dynamically (no SSR)
%    % % %  globals.css        # Luna Theme global styling variables & overrides
% % %  components/
%    % % %  XPDesktop.jsx      # Root state, Window registry, theme toggles, orchestrator
%    % % %  BootScreen.jsx     # HTML layout & CSS keyframe animation for the boot bar
%    % % %  LoginScreen.jsx    # Authentication interface (Firebase, Google, Guest)
%    % % %  Taskbar.jsx        # Bottom bar, window tabs, clock state, cursors
%    % % %  StartMenu.jsx      # Double-column explorer pop-up
%    % % %  WindowManager.jsx  # Registry map rendering active windows
%    % % %  XPWindow.jsx       # Custom chrome container (draggable title bar, resize grip)
% % %  lib/
    % % %  firebase.js        # Auth and Firestore read/write integrations
    % % %  portfolioData.js   # Predefined developer projects, skills, education
    % % %  PortfolioContext.jsx # React context sync for Firestore data edits
```

Chapter 2: Project Setup & Dependency Configuration

Next.js is configured using React 19 and the App Router. Since desktop elements (specifically window positioning and mouse events) are exclusively client-side browser APIs, components must execute dynamically and disable Server-Side Rendering (SSR). This prevents window/document-undefined compilation errors.

1. Initializing the Project

Initialize a standard Next.js directory and clean up default pages:

```
npx create-next-app@latest windows-xp-portfolio \
  --js \
  --eslint \
  --no-src-dir \
  --app \
  --no-tailwind \
  --import-alias "@/*"
```

2. Dynamic Imports Without SSR

In src/app/page.js, we import the core application dynamically. This forces Next.js to package the desktop features as client-only bundles, letting React bootstrap the page directly on the browser:

```
'use client';
import dynamic from 'next/dynamic';

const XPDesktop = dynamic(() => import('@/components/XPDesktop'), {
  ssr: false,
  loading: () => (
    <div style={{ background: '#000', width: '100vw', height: '100vh',
                  display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
      <div style={{ color: '#fff', fontFamily: 'Tahoma, sans-serif', fontSize: 13 }}>
        Loading Windows XP Portfolio...
      </div>
    </div>
  ),
});

export default function Home() {
  return <XPDesktop />;
}
```

3. Standard Dependencies

Install libraries required for authentication, database management, and animation:

```
npm install firebase gsap
```

- firebase: Used for authentication (Email, Google Auth) and storing portfolio data plus guest messages in

Cloud Firestore.

- gsap: GreenSock Animation Platform. Utilized for natural springy window entrance effects (back.out) and taskbar minimization transitions.

Chapter 3: Designing the Luna CSS System

The Luna design system replicates the blue theme of Windows XP. It uses specific color codes, gradients, and borders. Instead of heavy CSS libraries, vanilla CSS Modules are employed to keep components modular and styling clean.

1. Global Color Variables & System Border Reliefs

Define variables in src/app/globals.css to keep style tokens consistent across components. Win32 styling relies heavily on 3D relief box shadows to emulate button bevels:

```
:root {
  /* Classic XP Colors */
  --xp-blue-dark: #0058ee;
  --xp-blue-light: #3ca4ff;
  --xp-blue-titlebar: linear-gradient(90deg, #0d59f2 0%, #3ca4ff 100%);
  --xp-start-orange: linear-gradient(180deg, #388e3c 0%, #4caf50 100%);
  --xp-taskbar-bg: linear-gradient(180deg, #245dd7 0%, #0d3eb3 100%);
  --xp-background-desktop: #0058ee;

  /* Fonts */
  --xp-font-tahoma: "Tahoma", "Segoe UI", system-ui, sans-serif;

  /* 3D Border Effects */
  --xp-border-outset: 1.5px 1.5px 0px #ffffff inset, -1.5px -1.5px 0px #868a8e inset;
  --xp-border-inset: -1.5px -1.5px 0px #ffffff inset, 1.5px 1.5px 0px #868a8e inset;
  --xp-border-window: 3px solid #0058ee;
}
```

2. Recreating the Classic Win32 Scrollbars

Use standard CSS selectors to force scrollbars to render with retro XP gray colors and arrows:

```
.xp-scroll::-webkit-scrollbar {
  width: 16px;
  height: 16px;
  background: #f1f0ec;
}
.xp-scroll::-webkit-scrollbar-thumb {
  background: #d4d0c8;
  border: 2px solid #f1f0ec;
  box-shadow: var(--xp-border-outset);
}
.xp-scroll::-webkit-scrollbar-button {
  display: block;
  background: #d4d0c8;
  border: 1px solid #808080;
  box-shadow: var(--xp-border-outset);
}
```

Design Tip: The XP look depends on avoiding smooth modern rounded borders on window components. Ensure borders are strictly square (radius 0px) and buttons use the dual relief (highlight top-left, shadow bottom-right) to trigger spatial depth perception.

Chapter 4: The Boot and Login State Screens

The application initializes in the boot phase. Once complete, it fades into the account login screen.

1. Boot Screen Marching Bar (BootScreen.jsx)

The loading bar features three small blue gradient blocks moving inside a border wrapper. We implement this using a standard CSS translation keyframe loop:

```
/* CSS Animation for Marching Bar */
.progressWrap {
  width: 160px;
  height: 14px;
  border: 2px solid #5a5a5a;
  background: #000;
  position: relative;
  overflow: hidden;
}
.barGroup {
  display: flex;
  gap: 2px;
  position: absolute;
  width: 36px;
  animation: xp-march 1.8s linear infinite;
}
.segment {
  width: 8px;
  height: 100%;
  background: linear-gradient(180deg, #0d59f2 0%, #3ca4ff 50%, #0d59f2 100%);
}
@keyframes xp-march {
  0% { transform: translateX(-40px); }
  100% { transform: translateX(160px); }
}
```

2. Firebase Authentication UI (LoginScreen.jsx)

The login interface offers three methods: admin password validation, Google OAuth, and Guest mode. Clicking guest mode signs out any existing auth and moves to the desktop. This setup is managed inside the login component:

```
const handleEmailLogin = async () => {
  if (!password) { setError('Enter password'); return; }
  setLoading(true);
  try {
    // Admin log in is synchronized with Firebase Auth
    await loginWithEmail('kamal.bharadwj@gmail.com', password);
    showToast('Welcome back, Kamal!', 'success');
    handleFinish();
  } catch (err) {
    setError('Incorrect password. Try again.');
  } finally { setLoading(false); }
};

const handleGuestLogin = () => {
  showToast('Browsing as Guest', 'info');
  handleFinish(); // Triggers GSAP exit scale animation
};
```

Chapter 5: The Desktop & Navigation Shell

The desktop provides the wrapper for dragging, icons, and menus. It coordinates state for active applications, z-indexes, and mouse position trackers.

1. Centralized Window Registry (XPDesktop.jsx)

Windows are defined in a registry map. It holds window details, default dimensions, icons, and initial layouts:

```
const WINDOWS = {
  about:    { title: 'About Me — Portfolio.exe',         icon: '/icons/xp_about.svg',
defaultW: 680, defaultH: 500 },
  projects: { title: 'My Projects',                      icon: '/icons/xp_projects.svg',
defaultW: 780, defaultH: 520 },
  skills:   { title: 'My Computer — Skills & Resume',    icon: '/icons/xp_mycomputer.svg',
defaultW: 740, defaultH: 560 },
  contact:  { title: 'Contact Me',                       icon: '/icons/xp_contact.svg',
defaultW: 560, defaultH: 480 },
  search:   { title: 'Search Companion',                 icon: '/icons/xp_search.svg',
defaultW: 380, defaultH: 480 },
  admin:    { title: 'Portfolio Admin Panel',             icon: '/icons/xp_admin.svg',
defaultW: 850, defaultH: 600 },
};
```

2. Taskbar & clock (Taskbar.jsx)

The taskbar displays active applications. A `setInterval` timer updates the clock format in the bottom right corner:

```
const [time, setTime] = useState('');
useEffect(() => {
  const updateClock = () => {
    const d = new Date();
    let hrs = d.getHours();
    const mins = d.getMinutes().toString().padStart(2, '0');
    const ampm = hrs >= 12 ? 'PM' : 'AM';
    hrs = hrs % 12 || 12;
    setTime(`${hrs}:${mins} ${ampm}`);
  };
  updateClock();
  const timer = setInterval(updateClock, 60000);
  return () => clearInterval(timer);
}, []);
```

3. Double-Column Start Menu (StartMenu.jsx)

The Start Menu divides items into two lists: application short-cuts on the left, and system configurations and logout toggles on the right, decorated with the administrator's user avatar.

Interaction Detail: In the Start Menu, clicking "All Programs" displays an expandable sub-menu list. Clicking "Log Off" clears the active user session in Firebase and triggers a state fade back to the Login Screen.

Chapter 6: Dynamic Windowing & Draggability

The core of the desktop experience is the window wrapper (XPWindow.jsx). It must handle resizing, double-click actions, and dragging constraints to prevent elements from going off-screen.

1. Draggability Calculation

We implement dragging inside the title bar using mouse event listeners. We keep the top bar of the window within the visible screen area to prevent it from getting stuck:

```
const handleMouseDown = (e) => {
  if (maximized || isMobile) return;
  dragging.current = true;
  offset.current = { x: e.clientX - x, y: e.clientY - y };
  onFocus();
  e.preventDefault();
};

useEffect(() => {
  const onMouseMove = (e) => {
    if (dragging.current && !isMobile) {
      const TASKBAR_H = 30;
      // Clamping coordinates to browser viewport boundaries
      const nx = Math.max(0, Math.min(e.clientX - offset.current.x, window.innerWidth - w));
      const ny = Math.max(0, Math.min(e.clientY - offset.current.y,
                                      window.innerHeight - TASKBAR_H - 30));
      onMove(nx, ny);
    }
  };
  const onUp = () => { dragging.current = false; };
  window.addEventListener('mousemove', onMouseMove);
  window.addEventListener('mouseup', onUp);
  return () => {
    window.removeEventListener('mousemove', onMouseMove);
    window.removeEventListener('mouseup', onUp);
  };
}, [x, y, w, h]);
```

2. GSAP Animations for Minimize & Maximize

GSAP powers fluid scaling animations when minimizing windows to the taskbar:

```
useEffect(() => {
  if (minimized) {
    gsap.to(windowRef.current, {
      scale: 0.1,
      opacity: 0,
      y: window.innerHeight - y - 100,
      x: (window.innerWidth / 2) - x - 150,
      duration: 0.25,
      ease: 'power2.in',
      onComplete: () => setLocalActive(false)
    });
  } else {
    setLocalActive(true);
    // ... restore scale to 1 using back.out ease
  }
}, [minimized]);
```

Chapter 7: Application Content Windows

Each window renders a specific component. These sub-components read from centralized data stores and handle database reads/writes.

1. Projects Folder Layout (ProjectsWindow.jsx)

The projects window mimics the classic Windows Explorer folder view. It uses a grid to display projects as system folders or text documents. Clicking a folder opens its details in a modal:

```
export default function ProjectsWindow() {
  return (
    <div className={styles.explorer}>
      <div className={styles.sidebar}>
        <div className={styles.sysTasks}>Folder Tasks</div>
        <div className={styles.details}>Details Pane</div>
      </div>
      <div className={styles.grid}>
        {projects.map((p) => (
          <div key={p.id} className={styles.folderIcon} onDoubleClick={() => openProject(p)}>
            <img src="/icons/xp_folder.svg" alt="" />
            <span>{p.title}</span>
          </div>
        ))}
      </div>
    </div>
  );
}
```

2. Contact Submission (ContactWindow.jsx)

The contact form saves name, email, and message inputs to the Firestore database. Submitting triggers an XP dialog box popup:

```
const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    await submitContactMessage({ name, email, message });
    showToast('Message sent! I will respond soon.', 'success');
    setName(''); setEmail(''); setMessage('');
  } catch (err) {
    showToast('Error saving message. Please check internet connection.', 'error');
  }
};
```

3. Rover Search Dog Integration (SearchWindow.jsx)

The search window features an animated search companion dog (Rover) on the left side of the window, and a search input text box on the right. Typing a query performs a full-text search across all portfolio data (Projects, Skills, Bio) and lists matching results dynamically.

Chapter 8: Firebase Rules & Security Setup

The application uses Firebase Authentication to manage administrative write actions and Firestore Security Rules to protect user submissions.

1. Firestore Collections Structure

- portfolio: Holds a single document "data" with keys: bio, projects, education, experience, and skills. Read permissions are public; writes are restricted.
- contacts: Holds individual contact submissions with keys: name, email, message, and timestamp. Create permissions are public; read/delete permissions are restricted to the administrator.

2. Firestore Security Rules

Define rules to restrict access. Apply these in the Firebase Console:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Portfolio Data: Public reads, Admin-only writes
    match /portfolio/{doc} {
      allow read: if true;
      allow write: if request.auth != null &&
                      request.auth.token.email == 'kamal.bharadwj@gmail.com';
    }
    // Contacts: Anyone can submit, Admin-only reads/deletes
    match /contacts/{doc} {
      allow create: if true;
      allow read, delete: if request.auth != null &&
                             request.auth.token.email == 'kamal.bharadwj@gmail.com';
    }
  }
}
```

3. Deployment Setup

Build the project for production and deploy hosting config using firebase-tools CLI:

```
# 1. Build Next.js project
npm run build

# 2. Deploy static out directory to Firebase Hosting
npx firebase deploy
```

For Vercel deployment, link the project repository directly on the Vercel dashboard and add the `.env.local` Firebase keys as environment variables under Project Settings.

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar

Created by Kamal Kumar